MEETING EDITION / QUICK READ: PAGES 1-9

Implemented system architecture

Medical assistance grounded in retrieved video evidence

Medix is a Next.js chat application backed by FastAPI. It retrieves medical video passages, localizes a useful interval, packages evidence, and produces source-linked medical guidance. The website performs an additional conversational generation and verification stage after the backend response. [S1-S5]

Three facts to state accurately: Qdrant is absent from the active path. A photo alone receives a clarification. The project uses retrieval and prompting with existing models; it does not fine-tune Qwen.

Component	Implemented responsibility
Next.js 16 / React / AI SDK	Input, session/history, intent routing, conversational composition, streaming and evidence UI.
FastAPI / Uvicorn	Canonical text and multimodal APIs; safety triage; retrieve, localize, extract and validate evidence.
Voyager Qwen	ASU gateway, default **qwen35-27b**. Separate calls perform text reasoning and bounded visual observations.
Local retrieval models	MiniLM text embeddings and lexical matching; CLIP scores retrieved video frames against the question.
State and resources	PostgreSQL chat state; JSON catalogs/transcripts; local videos and generated evidence artifacts. Redis is optional.

Review scope and navigation

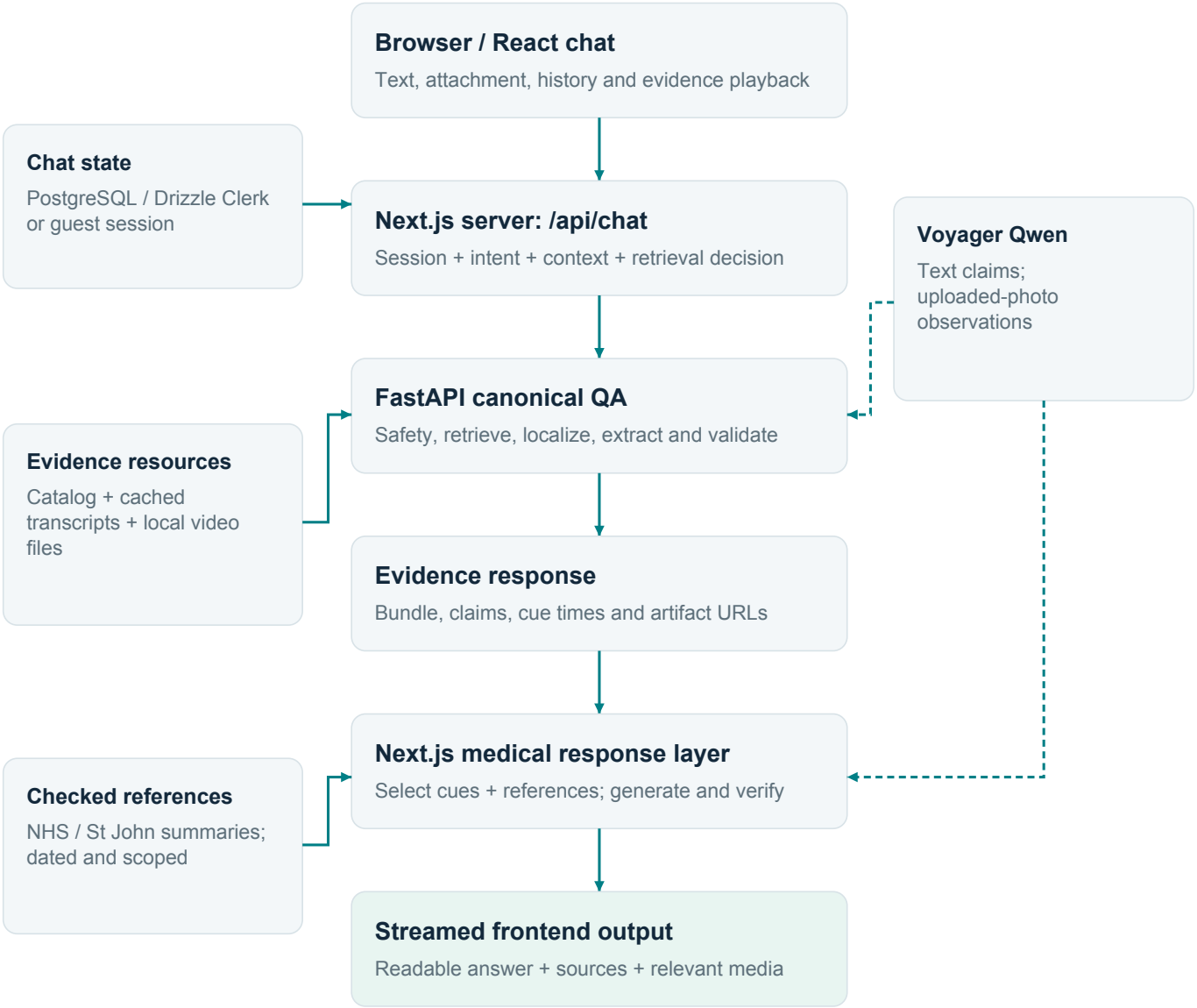
Inspected the active checkout at `/Users/sairithwikk/Documents/Medix`, including its 55 backend and 189 frontend source files by inventory, with targeted tracing of the active request paths, adapters, evidence contracts and launch script. Dataset and development tooling are distinguished from production components.

Read	Purpose
2-5	Overall diagram and the three input-mode flows
6-7	Grounding, models, storage, deployment and implementation map
8-9	Five-minute presentation guide and three demo cases
10-11	Editable Mermaid source and code references
12	Exact local and Sol launch commands

ARCHITECTURE BRIEF

1. Overall system and data flow

Solid arrows show primary handoffs; dashed arrows show model-provider calls. Side arrows supply state or evidence. The two generation stages have different purposes. [S2-S5]



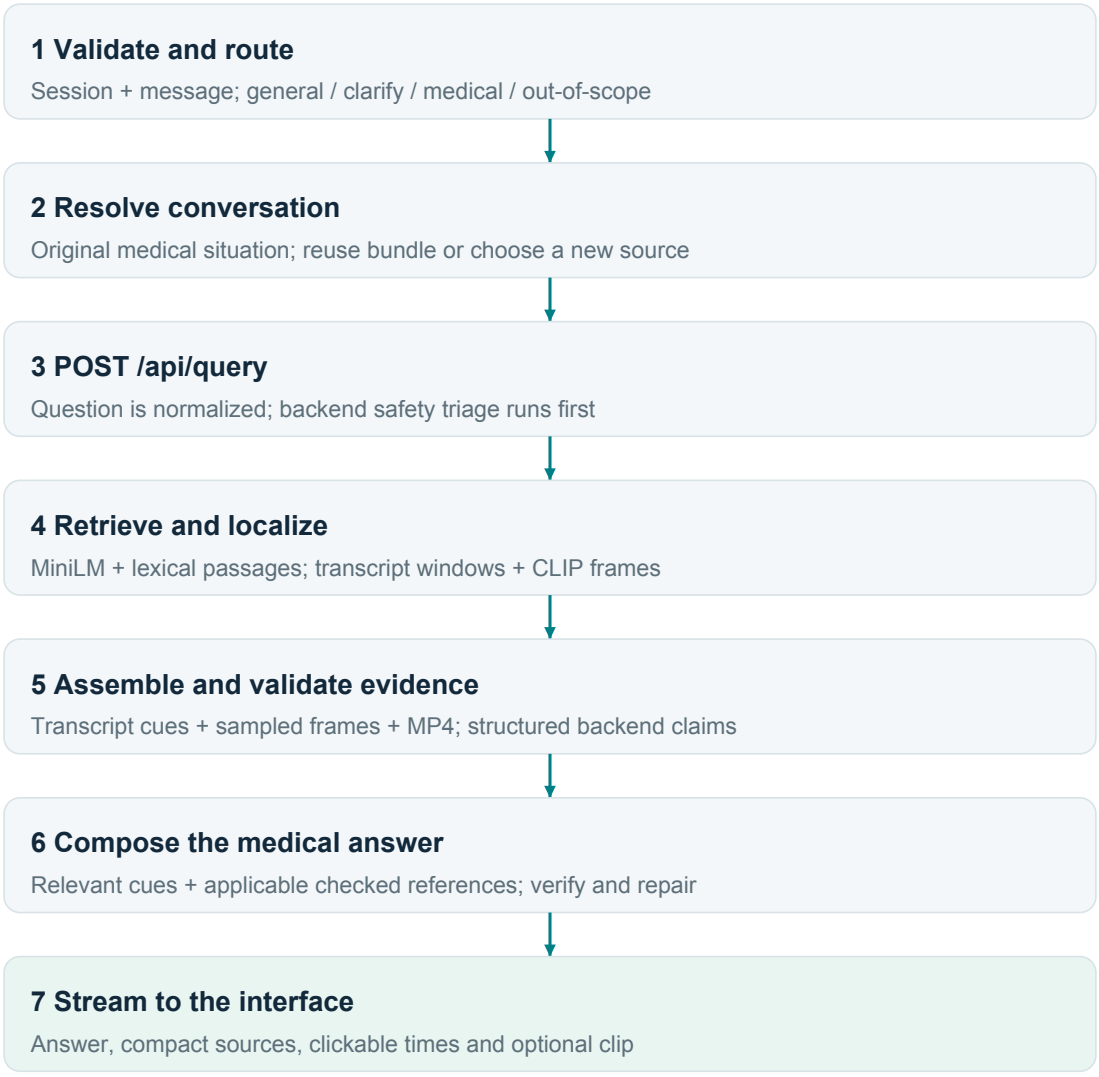
Edit this diagram in Mermaid Live | 01-overall.mmd

Same-topic follow-ups can bypass new backend retrieval and reuse the current bundle. A new image forces fresh retrieval. Video is shown only when directly relevant evidence survives selection. The backend response is not simply printed verbatim.

Qdrant: no active adapter, dependency or vector service call was found. Retrieval uses normalized NumPy embeddings and in-process lexical structures, not a running Qdrant instance.

ARCHITECTURE BRIEF

2. Text-only question workflow



Edit this diagram in Mermaid Live | 02-text.mmd

• **Routing:** greetings and Medix capability questions avoid procedure retrieval. Missing essential context prompts a short clarification; unrelated requests are redirected.

• **History:** recent text is bounded to roughly 3,000 characters; the context builder uses the latest eight messages and searches backward for the current evidence result.

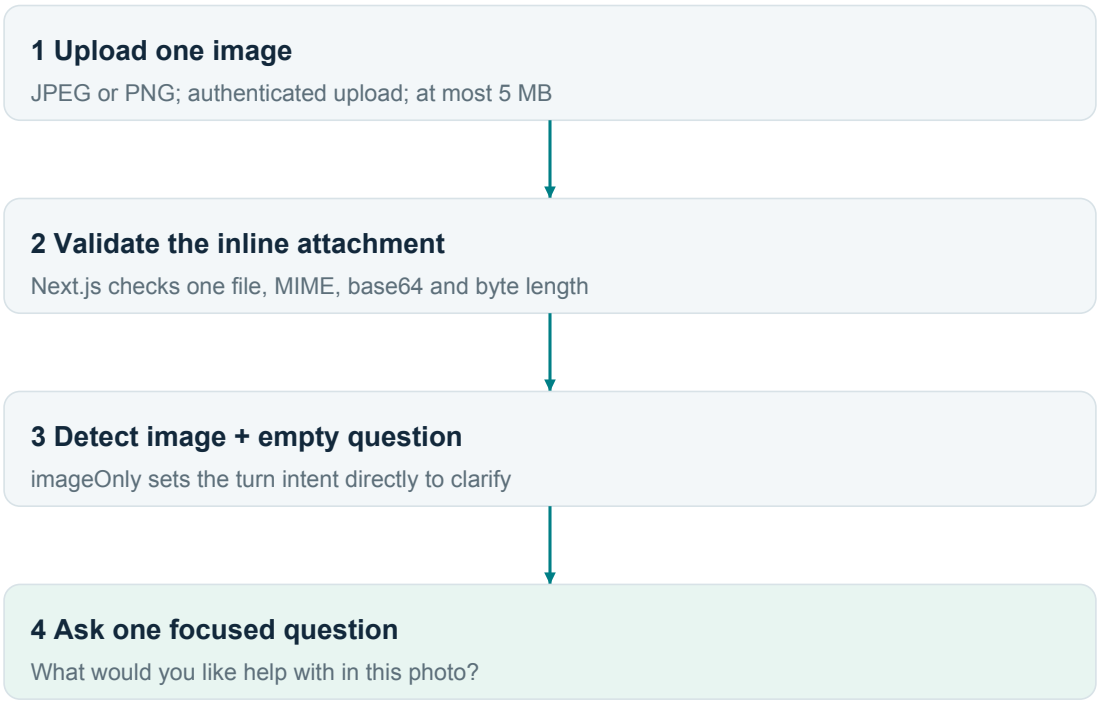
• **Text-only still uses video vision:** the active multimodal localizer runs CLIP on retrieved video frames. It does not call uploaded-photo observation when there is no image.

Code trace: `api/chat/route.ts -> search-procedure.ts -> /api/query -> GroundedMedicalQAPipeline.` [S2-S5]

ARCHITECTURE BRIEF

3. Image-only input: clarification first

The website accepts a photo without text, but it does not choose a medical procedure or diagnose the image automatically. This is a deliberately shorter path. [S2, S6]



Edit this diagram in Mermaid Live | 03-image-only.mmd

No medical inference on this turn: no Voyager visual observation, backend medical search, transcript retrieval or video attachment is initiated by the image-only branch.

What happens when the user continues?

The user supplies a specific medical question. To exercise the vision workflow, attach the photo together with that question: the medical route extracts image bytes from the latest user message only. A text reply alone does not automatically resend an earlier photo.

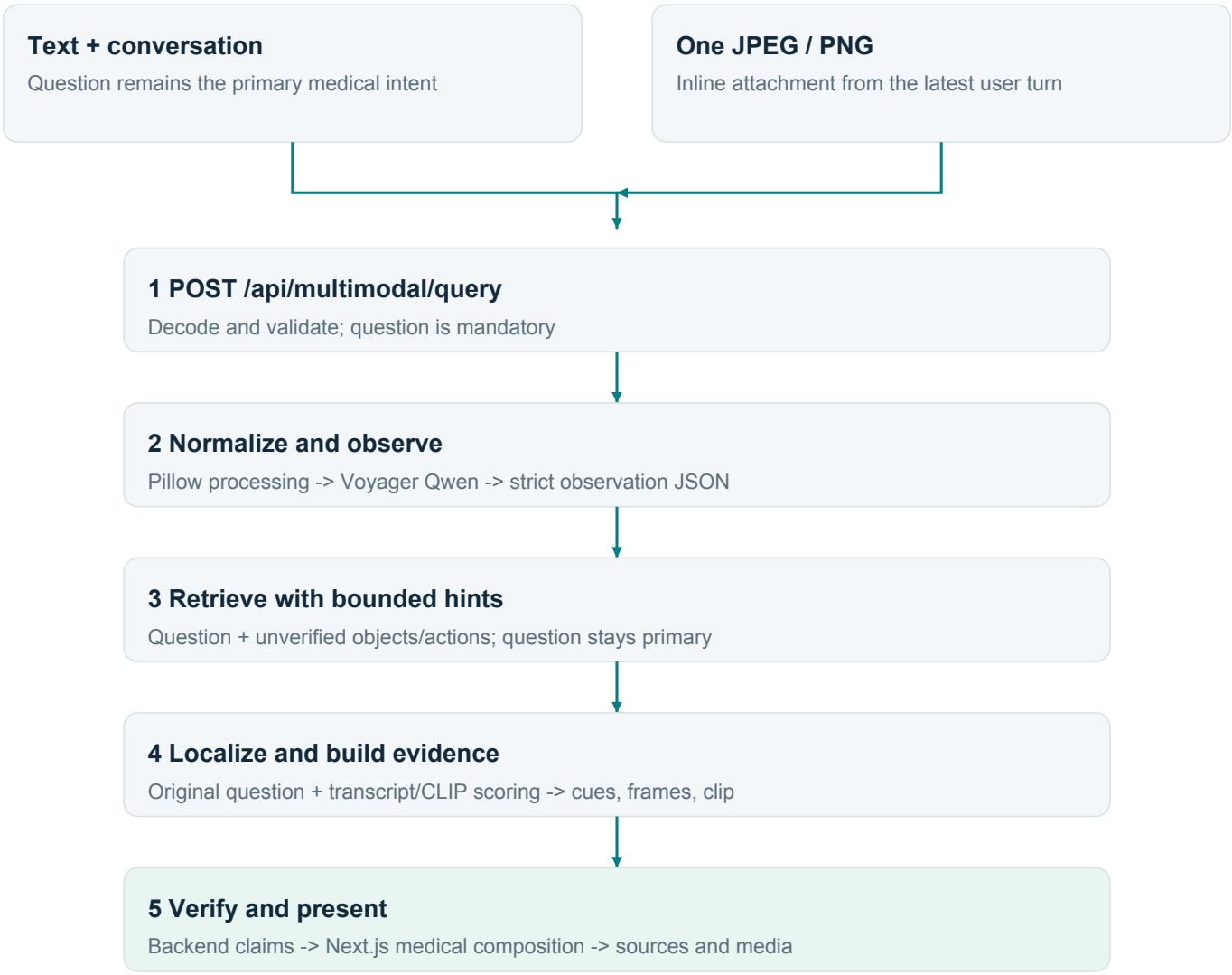
API boundary

`POST /api/multimodal/query` requires a non-empty `query` and `image_base64`. It is not an image-only diagnostic API. Blank questions fail the backend schema instead of triggering a hidden default question.

The upload endpoint returns an inline data URL. The active upload route does not store images in Vercel Blob or fetch arbitrary image URLs. Normal chat persistence still saves message parts in PostgreSQL. [S6, S10]

ARCHITECTURE BRIEF

4. Text + image: bounded visual context



Edit this diagram in Mermaid Live | 04-text-image.mmd

Boundary	Actual behavior
Image preprocessing	Backend: at most 10 MiB and 20 megapixels; one frame; EXIF orientation corrected; RGB; thumbnail within 1024 x 1024; JPEG quality 85; metadata removed. Website limit remains 5 MB.
Observation contract	Only visible_objects, visible_actions and uncertainty; bounded lists/strings. Images do not establish circulation, infection, severity or diagnosis.
Retrieval fusion	Objects/actions augment the retrieval query; passage ranking isolates the original text question. Localization and backend text generation use the original question.

Uploaded-image Qwen observations and CLIP scoring of retrieved video frames are separate operations. Sampled video frames are stored and scored; the canonical backend text generator receives transcript cues, not raw video frames. [S3-S7]

ARCHITECTURE BRIEF

5. Retrieval, grounding and response contract

Evidence acquisition and ranking

The local snapshot has **852 catalog documents**, **51 supported video IDs** and **61 cached transcript JSON files**. These counts describe local resources, not medical coverage or accuracy. The enabled supported-collection filter restricts eligible videos. [S3, S8]

Stage	Current live implementation
Video retrieval	MiniLM title/author embeddings plus lexical overlap; overlapping three-cue transcript groups use MiniLM dense scores and TF-IDF term coverage. No answer-label or gold-timestamp retrieval.
Passage fusion	Per passage: 0.5 x lexical coverage + 0.3 x dense similarity; best passage per video + 0.2 x metadata score. If lexical features are unavailable, use 0.8 x dense.
Temporal localization	Configured 90-second windows / 45-second stride; live boundaries expand to complete cues. Up to two transcript-ranked windows per video, three sampled frames per window.
Visual and joint scores	Current live defaults: 0.8 transcript + 0.2 CLIP frame score; video/window joint rank uses effective retrieval weight at least 0.5. Scores are rankings, not calibrated medical probabilities.
Evidence bundle	Question, predicted source interval, transcript cues, three sampled frame artifacts, extracted clip, provenance and content hashes. FFmpeg requires local video bytes.

Two verification layers

• **Backend:** Qwen returns structured transcript-cited claims. Bounded claim text, known cue IDs and lexical support are validated. It may try one other already-ranked source; invalid or missing evidence can produce an abstention.

• **Next.js:** select a minimal relevant cue set; retrieve applicable fixed medical summaries; generate conversational claims; verify support, relevance, scope and completeness. At most one repair cycle follows an incomplete draft.

• **Presentation:** T IDs become clickable video timestamps; K IDs become compact medical source links. Unsupported medical details cannot borrow a video timestamp. A previous verified partial answer may survive a failed repair.

Live timestamps are **model-predicted evidence intervals**, not guaranteed dataset annotation boundaries. Source-video and clip-relative coordinates are both returned; the UI preserves fractional seek coordinates. [S5, S9]

ARCHITECTURE BRIEF

6. Deployment, storage and code map

Location	Purpose and concrete implementation
frontend/	Next.js UI, /api/chat orchestration, uploads, auth, PostgreSQL/Drizzle, optional Redis streams. Next.js also makes its own Voyager calls.
backend/	FastAPI, ServiceFactory, QA service, in-memory retrieval, CLIP localizer, evidence extraction, Voyager adapters and local Kokoro speech.
data/ + MedVidQA/	Runtime catalog/transcripts/videos/artifacts versus original research dataset. Development tools and evaluations live under development/.
Local or Sol compute	The same FastAPI API can run locally or in a CPU Slurm job. The shipped script requests 4 CPUs, 16 GB, public partition, class QoS and 2 hours; no GPU is requested.
External services	Voyager supplies Qwen inference; PostgreSQL supplies chat state. Clerk handles configured registered accounts; Auth.js supports guest/legacy sessions. Qdrant is not required.

API / module	Responsibility
POST /api/query	Canonical text evidence response; /api/v2/query is a compatibility alias.
POST /api/multimodal/query	Question + validated image observations; image_context_is_evidence is false.
GET /api/artifacts/{path}	Serves generated clips/evidence inside the artifact root.
GET /api/health; POST /api/tts	Readiness; optional local Kokoro WAV synthesis, separate from medical grounding.
medical-follow-up.ts / medical-knowledge.ts	Website composition/verifier and five dated, scope-restricted reference summaries.

Demo startup assumptions

Commands on page 12 use the existing prepared Mac environment and private backend/.env / frontend/.env.local files; they do not provision credentials, databases or evidence. PostgreSQL and migrated chat tables must already be available. Run only one frontend and one backend per selected port; stop an older owned server with Ctrl+C first.

For Sol, the batch script activates the existing `medix` environment, loads cached models, builds the supported collection and starts port 5001. Use the printed compute node, not the login node. Wait for RUNNING before the node lookup/tunnel commands; PENDING is a scheduler wait. Account name on page 12 is from the supplied account association. Allocation validity and Sol availability must be confirmed at execution.

Operational limits: the active config uses local video files and cached transcripts. An OnDemandVideoRepository exists but is not enabled by the inspected runtime defaults. Qwen access alone cannot replace missing clips. Optional Ollama, Sol-model and CCAL paths are research alternatives, not the selected live path.

ARCHITECTURE BRIEF

7. Your five-minute presentation guide

Use the main diagrams rather than reading source code. The first nine pages are the meeting brief; Mermaid and commands are appendices.

Time	Show	Exactly what to explain
0:00-0:40	Page 1	"Medix connects medical questions to video evidence and checked references. It is a research assistant, using RAG with existing Qwen models."
0:40-1:40	Diagram 1, page 2	Trace browser -> Next.js -> FastAPI -> evidence -> conversational verification -> UI. Point out PostgreSQL, Voyager, and local evidence resources. State that Qdrant is not active.
1:40-2:30	Diagram 2, page 3	Explain MiniLM/lexical retrieval, temporal localization, CLIP frame reranking, and a transcript-cited evidence bundle. Explain reuse of a current bundle for follow-ups.
2:30-3:15	Diagrams 3 and 4, pages 4-5	Photo alone prompts clarification. Photo + question gets normalized and observed by Qwen; observations guide retrieval but are not clinical proof. CLIP handles retrieved video frames separately.
3:15-4:10	Page 6; then demo 1 or 3	Contrast backend structured grounding with the website conversational verifier. Click one timestamp; compare medical reference links with video support.
4:10-5:00	Page 7; demo summary on page 9	Explain local versus Sol hosting, CPU evidence work versus Voyager inference, resource prerequisites and current limits. End with what you plan to evaluate next.

Three likely professor questions

• **Is it fine-tuned on MedVidQA?** No fine-tuning is implemented in the live path. The dataset supplies a retrieval/evaluation resource; the runtime builds evidence context for existing models.

• **Does the model understand the video?** CLIP samples video frames for relevance; generation primarily uses transcript cues. Uploaded photos have a separate bounded Qwen observation step. This is not end-to-end video-language reasoning over the full clip.

• **How is reliability established?** Provenance, claim checks and regression tests reduce specific failure modes; they do not establish clinical accuracy. The generator/verifier share a provider, and source coverage is limited.

Meeting shortcut: if time is tight, present pages 2, 4 and 5, then run the text demo and one timestamp click. Keep page 12 ready for startup; do not attempt a new model/data download during the meeting.

ARCHITECTURE BRIEF

8. Three realistic demo test cases

Demo 1 | Text: bandage check and continuity

Question: "I just put a bandage on someone's arm. How do I know if it's too tight?"
Follow-up: "What if the colour doesn't come back quickly?"

Expected: explain the circulation check, its timing/interpretation and supported next action, with relevant warning signs where applicable. The follow-up should answer what to do next using the prior situation. Only directly relevant moments should appear; wording and completeness can vary.

Demonstrate: direct answer, a compact NHS/St John link, one appropriate timestamp and same-topic evidence reuse. The cached severe-bleeding transcript `NxO5LvgqZe0` contains the nail/skin check; retrieval is not guaranteed to select that video.

Demo 2 | Image only: safe clarification

Input: attach one non-identifying JPEG/PNG of a bandaged forearm, leave the message empty and send.

Expected: "What would you like help with in this photo?" No procedure answer, severity judgement, diagnosis or automatic evidence clip should be produced on this turn.

Demonstrate: the attachment control and the clarification-first branch. This case tests an intentional limitation, not a failed image model. For the next demo, reattach the image together with the question.

Demo 3 | Text + image: observations versus evidence

Input: the same photo plus "I just put this bandage on someone's arm. How can I check whether it's too tight?"

Expected: image normalization, bounded visual-object/action observations and medical retrieval. The response may use the visible bandage as context, but must not claim to determine circulation or tightness from the photo alone. Guidance needs transcript or checked-reference support.

Demonstrate: visible upload thumbnail, source-linked care instructions, relevant evidence only and the distinction between a reference link and a video timestamp. Unsupported or unavailable evidence can produce a partial answer or an honest limitation.

Before the meeting: verify /api/health, confirm Voyager access and PostgreSQL, use a small non-identifying image, and preload the page. Do not present a generated answer as a clinical validation result. These are expected behaviors, not a claim that an actual image run was validated in this documentation task.

MERMAID SOURCE / OVERALL AND TEXT-ONLY

Appendix A. Editable Mermaid diagrams

The print diagrams are vector layouts of these same components and connections. Use the links to open preloaded editable Mermaid diagrams, or paste the .mmd source into Mermaid Live Editor.

A1 | Overall system

Edit this diagram in Mermaid Live | 01-overall.mmd

```
flowchart TD
 U[Browser: text or image] --> N[Next.js /api/chat]
 P[(PostgreSQL chat history)] <--> N
 N --> R{Route and resolve context}
 R -->|clarify or general| O[Streamed chat output]
 R -->|new medical question| F[FastAPI canonical QA]
 R -->|same-topic follow-up| C[Next.js response composer]
 D[(Catalog + transcripts + local videos)] --> F
 V[Voyager Qwen: text and vision] <--> F
 F --> E[Evidence bundle + validated backend claims]
 E --> C
 K[Checked NHS / St John references] --> C
 V <--> C
 C --> O
 E --> M[Optional relevant clip + transcript]
 M --> U
 O --> U
```

A2 | Text-only request

Edit this diagram in Mermaid Live | 02-text.mmd

```
flowchart TD
 A[Text question] --> B[Validate session and message]
 B --> C[Intent + conversation context]
 C -->|new source| D[POST /api/query]
 C -->|same topic| I[Reuse current evidence]
 D --> E[Safety triage]
 E --> F[MiniLM + lexical passage retrieval]
 F --> G[Transcript windows + CLIP frame scoring]
 G --> H[Extract cues, frames and clip]
 H --> J[Voyager structured backend claims]
 J --> I
 I --> K[Select relevant cues + checked references]
 K --> L[Generate, verify and bounded repair]
 L --> M[Chat + inline sources + optional video]
```

Source files and editor URLs are saved under `development/docs/architecture-diagrams/`. Live-editor links contain diagram text only, not credentials or application data.

MERMAID SOURCE / IMAGE MODES AND TRACEABILITY

Appendix B. Mermaid and source evidence

B1 | Image-only clarification

Edit this diagram in Mermaid Live | 03-image-only.mmd

```
flowchart TD
 A[One JPEG or PNG, no text] --> B[Validate upload and inline bytes]
 B --> C[Next.js detects empty question + image]
 C --> D[Deterministic clarification reply]
 D --> E[What would you like help with in this photo?]
 E --> F[User provides a specific question]
 F --> G[Reattach photo with question for vision]
 G --> H[Text + image workflow]
 C -.-> X[No vision call or RAG for this turn]
 C -.-> Y[No diagnosis or automatic procedure choice]
```

B2 | Text + image

Edit this diagram in Mermaid Live | 04-text-image.mmd

```
flowchart TD
 T[Text question + history] --> J[Next.js route and contextualize]
 I[One inline JPEG or PNG] --> J
 J --> A[POST /api/multimodal/query]
 A --> P[Pillow: validate, orient, resize, strip EXIF]
 P --> V[Voyager Qwen visual observation]
 V --> O[Validated objects / actions / uncertainty]
 O --> Q[Question + unverified retrieval hints]
 Q --> R[MiniLM + lexical catalog / transcript retrieval]
 T --> L[Original question for localization and QA]
 R --> L
 L --> C[CLIP video frames + temporal localization]
 C --> E[Transcript cues + frames + extracted clip]
 E --> B[Backend structured text claims + validation]
 B --> G[Next.js medical-reference generation + verification]
 O -.->|context, not clinical evidence| G
 G --> U[Streamed answer + precise relevant evidence]
```

Implementation references (relative to project root)

[S1] README.md; pyproject.toml; frontend/package.json
[S2] frontend/app/(chat)/api/chat/route.ts; lib/ai/chat-routing.ts
[S3] backend/container.py; backend/research_pipeline.py; backend/config.py
[S4] backend/video_retrieval/{semantic,transcript}.py; temporal_localization/
[S5] backend/services/{end_to_end,grounded_qa}.py; evidence/extractor.py
[S6] frontend/lib/ai/image-input.ts; api/files/upload/route.ts; backend/api/routes.py
[S7] backend/adapters/{voyager,structured_vision,vision_context,visual_similarity}.py
[S8] data/medvidqa_video_catalog.json; supported_video_collection.json; transcript_cache/
[S9] frontend/lib/ai/medical-follow-up.ts; medical-knowledge.ts; lib/evidence-timestamps.ts
[S10] frontend/lib/dev-session.ts; lib/db/queries.ts; components/chat/procedure-card.tsx
[S11] backend/medix_live_backend.sbatch; backend/config.example.yaml; app.py

PREPARED ENVIRONMENTS / TERMINAL COMMANDS ONLY

Appendix C. Exact launch commands

Local Mac | backend terminal

```
cd /Users/sairithwikk/Documents/Medix
source /Users/sairithwikk/.cache/medix/venv/bin/activate
HF_HUB_OFFLINE=1 TRANSFORMERS_OFFLINE=1 \
python -m uvicorn backend.app:app --host 127.0.0.1 \
  --port 5001 --env-file backend/.env
```

Local Mac | frontend terminal

```
cd /Users/sairithwikk/Documents/Medix/frontend
MEDIX_API_URL=http://127.0.0.1:5001 pnpm dev
```

Sol backend | submit, inspect RUNNING node, then tunnel from Mac

```
LIVE_JOB=$(ssh skukunu1@sol.asu.edu \
  'cd ~/Medix && module purge && module load mamba/latest &&
   source activate medix &&
   sbatch --parsable --account=class_cse57268549fall2026 \
   backend/medix_live_backend.sbatch')
ssh skukunu1@sol.asu.edu \
  "squeue -j $LIVE_JOB -o '%.18i %.8T %.20N'"
ssh skukunu1@sol.asu.edu \
  "tail -n 30 ~/Medix/medix-live-$LIVE_JOB.out"
LIVE_NODE=$(ssh skukunu1@sol.asu.edu \
  "squeue -h -j $LIVE_JOB -t R -o %N")
test -n "$LIVE_NODE" && ssh -N \
  -L "5002:$LIVE_NODE:5001" skukunu1@sol.asu.edu
```

Sol backend | separate Mac frontend terminal

```
curl -fsS http://127.0.0.1:5002/api/health
cd /Users/sairithwikk/Documents/Medix/frontend
MEDIX_API_URL=http://127.0.0.1:5002 pnpm dev
```